deviceDiscovery MQTT/TLS/JSON

Interface Design Description

Service ID: *"deviceDiscovery"*

Abstract
This document describes the MQTT protocol with TLS payload security and JSON payload encoding variant of the `deviceDiscovery` service of the ServiceRegistry system.

Contents

1 Documentation edition and implementation scope **3**

2 Overview **3**

3 Service Operations **4**
3.1 operation register . . . 4
3.2 operation lookup . . . 4
3.3 operation revoke . . . 5

4 Data Models **6**
4.1 struct MqttRequestTemplate . . . 6
4.2 struct MqttResponseTemplate . . . 6
4.3 struct ErrorMessage . . . 6
4.4 struct Address . . . 6
4.5 struct DeviceListResponse . . . 6
4.6 struct DeviceLookupRequest . . . 7
4.7 struct DeviceRequest . . . 7
4.8 struct DeviceResponse . . . 7
4.9 struct MetadataRequirement . . . 7
4.10 Primitives . . . 7

5 Go and Rust implementation variants **9**
5.1 Go implementation . . . 9
5.2 Rust implementation . . . 9

6 References **9**

7 Revision History **10**
7.1 Amendments . . . 10
7.2 Quality Assurance . . . 10

1 Documentation edition and implementation scope

This is the 5.3.0 documentation edition, dated 2026-11-09. It carries forward the system and service contracts from the retained 5.2.2 documentation and identifies the current primary implementation sources. The former `documentation/v5.2.2` folder is retained unchanged.

Document version 5.3.0 is distinct from executable, dependency, registered system/service and wire-contract versions. Values still defined by the source (including 5.2.2 artefact versions, 1.0.0 service versions and `SysMonitor-AMQP-IDD-5.2.2` payload identifiers) are preserved. This documentation update does not change runtime behaviour or establish a new implementation release. Historical verification statements describe the tests recorded in the inherited baseline; this revision validates document compilation and references.

2 Overview

This document describes the `deviceDiscovery` service interface of the ServiceRegistry system over MQTT, which provides registration and lookup of the devices of the local cloud: the physical or virtual hosts, with their network addresses and metadata, on which the systems run.

It's implemented using protocol, encoding as stated in below table:

Profile type	Type	Version
Transfer protocol	MQTT	3.1.1 [1]
Data encryption	TLS	1.3 [2], 1.2 [3]
Encoding	JSON	RFC 8259 [4]
Compression	N/A	-
Semantics	N/A	-
Ontology	N/A	-

Table 1: Communication and semantics details used for the `deviceDiscovery` service interface

This document provides the Interface Design Description IDD to the *deviceDiscovery – Service Description* document [5]. The operations, their semantics and the data models are the same as in the HTTP/TLS/JSON IDD [6]; this document specifies the MQTT binding.

The MQTT interface is offered only when the system runs with `mqtt.api.enabled=true`; it is then registered with the interface template `generic_mqtts` (`generic_mqtt` without TLS), the broker address and port (`mqtt.broker.address`, `mqtt.broker.port`) and the base topic `arrowhead/serviceregistry/device-discovery/`. In the Arrowhead-test-cloud-java deployment the MQTT interfaces of the core systems are disabled.

3 Service Operations

The provider subscribes to one topic per operation, the base topic followed by the operation name:

Operation	Request topic
`register`	`arrowhead/serviceregistry/device-discovery/register`
`lookup`	`arrowhead/serviceregistry/device-discovery/lookup`
`revoke`	`arrowhead/serviceregistry/device-discovery/revoke`

A consumer publishes a request envelope (MqttRequestTemplate) to the topic and receives the answer (MqttResponseTemplate) on its own `responseTopic`. The `authentication` field carries the requester's identity according to the provider's authentication policy: `IDENTITY-TOKEN//<token>` (outsourced or internal policy), `SYSTEM//<name>` (declared policy) or the requester's X.509 certificate (certificate policy). The `status` of the response uses the HTTP status code numbers (200, 201, 204, 400, 401, 403, 404, 408, 423, 500, 503); on error the payload is an ErrorMessage.

```
{
  "traceId": "a1b2c3",
  "authentication": "IDENTITY-TOKEN//<token>",
  "responseTopic": "arrowhead/response/TemperatureDisplay/a1b2c3",
  "qosRequirement": 1,
  "params": {},
  "payload": {}
}
```

Listing 1: Request envelope.

```
{
  "status": 200,
  "traceId": "a1b2c3",
  "receiver": "TemperatureDisplay",
  "payload": {}
}
```

Listing 2: Response envelope.

3.1 PUBLISH `arrowhead/serviceregistry/device-discovery/register`

Operation: register
Input: DeviceRequest
Output: DeviceResponse

Registers a device with its addresses and metadata. If a device with the same name and the same data exists, it is returned unchanged (200); a new device is created (201). A device with the same name and different data is refused.

```
{
  "name": "Raspberry1",
  "metadata": {
    "location": "hall-1"
  },
  "addresses": [
    "string"
  ]
}
```

Listing 3: Payload of an register request.

3.2 PUBLISH `arrowhead/serviceregistry/device-discovery/lookup`

Operation: lookup
Input: DeviceLookupRequest
Output: DeviceListResponse

Returns the devices matching all given filters; without filters all devices.

```
{
  "deviceNames": [
    "Raspberry1"
  ],
  "addresses": [
    "string"
  ],
  "addressType": "IPV4",
  "metadataRequirementList": [
    {
      "unit": "celsius"
    }
  ]
}
```

Listing 4: Payload of an lookup request.

3.3 PUBLISH `arrowhead/serviceregistry/device-discovery/revoke`

Operation: revoke
Input: String

Removes the named device. Returns 200 if it was removed, 204 if it did not exist, and 423 Locked if systems are still assigned to it.

```
"TemperatureSensor"
```

Listing 5: Payload of an revoke request.

4 Data Models

Here, all data objects that can be part of the service calls associated with this service are listed in alphabetic order. Each *struct* denotes a JSON Object with the named fields; fields that are not needed may be omitted.

4.1 struct MqttRequestTemplate

The request envelope.

Object Field	Value Type	Description
"traceId"	String	Identifier chosen by the requester, returned in the response.
"authentication"	String	Identity of the requester (see Section 3).
"responseTopic"	String	Topic on which the response is published.
"qosRequirement"	Number	MQTT QoS of the response (0, 1 or 2).
"params"	Object<String>	Optional parameters (the query parameters of the HTTP binding).
"payload"	Object	The operation input.

4.2 struct MqttResponseTemplate

The response envelope.

Object Field	Value Type	Description
"status"	Number	Status code.
"traceId"	String	The trace identifier of the request.
"receiver"	String	Name of the requester.
"payload"	Object	The operation output, or an ErrorMessage.

4.3 struct ErrorMessage

Object Field	Value Type	Description
"errorMessage"	String	
"errorCode"	Number	
"exceptionType"	ExceptionType	
"origin"	String	

4.4 struct Address

A network address with its type.

Object Field	Value Type	Description
"type"	String	Address type: `HOSTNAME`, `IPV4`, `IPV6` or `MAC`.
"address"	String	One network address (host name, IPv4, IPv6 or MAC address).

4.5 struct DeviceListResponse

List of devices.

Object Field	Value Type	Description
"entries"	Array<DeviceResponse>	Matching records of the current page.
"count"	Number	Total number of matching records (all pages).

4.6 struct DeviceLookupRequest

Filter for devices; all fields optional, combined with AND.

Object Field	Value Type	Description
"deviceNames"	Array<String>	Device names to match.
"addresses"	Array<String>	Network addresses of the entity.
"addressType"	String	Address type filter: `HOSTNAME`, `IPV4`, `IPV6` or `MAC`.
"metadataRequirement List"	Array<MetadataRequirement>	Metadata requirements; each map is one alternative.

4.7 struct DeviceRequest

A device and its addresses.

Object Field	Value Type	Description
"name"	String	Device name, PascalCase.
"metadata"	Object	Free-form metadata (JSON object).
"addresses"	Array<String>	Network addresses of the entity.

4.8 struct DeviceResponse

A registered device.

Object Field	Value Type	Description
"name"	String	Device name.
"metadata"	Object	Free-form metadata (JSON object).
"addresses"	Array<Address>	Network addresses with their types.
"createdAt"	DateTime	UTC time of creation.
"updatedAt"	DateTime	UTC time of the last change.

4.9 struct MetadataRequirement

A JSON object of metadata requirements: each key is a metadata path (dot notation), each value the required value or an operator object such as `{"op": "LESS_THAN", "value": 10}`.

4.10 Primitives

As all messages are encoded using the JSON format [4], the following primitive constructs, part of that standard, become available.

Type	Description
Array	A JSON Array [4]; Array<A> has elements of type A.
DateTime	JSON String with an ISO 8601 UTC timestamp, e.g. `2026-10-22T12:00:00Z`.
ExceptionType	One of `ARROWHEAD`, `INVALID_PARAMETER`, `AUTH`, `FORBIDDEN`, `DATA_NOT_FOUND`, `TIMEOUT`, `LOCKED`, `INTERNAL_SERVER_ERROR`, `EXTERNAL_SERVER_ERROR` (JSON String).
Number	JSON Number, an integer.
Object	JSON Object; Object<A> has string keys and values of type A.
String	JSON String.

5 Go and Rust implementation variants

5.1 Go implementation

Primary system folder: `Eclipse-Arrowhead-go/ah5-core-go/serviceregistry`.

The system owns its Go module, launcher and build configuration. Use the per-system `Makefile` and `go.mod`; the containing Go tree is the build source, not a historical Arrowhead copy. Ordinary native roles use independent identities and JSON snapshot state rather than the Java database. Do not assume Java SQL migration, clustering or identical transaction/retry semantics. The registry has its own configuration and optional native AMQP monitor; other roles must be checked against their own registrations.

Listening port. Default 8443 (server.port in application.properties). Override with server.port, SERVER_PORT=N or -server.port N; the host is server.address. The native registry API, outsourced authentication, JSON persistence, optional AMQP telemetry and differences from Java are specified in `Eclipse-Arrowhead-go/ah5-core-go/serviceregistry/UPSTREAM-README.md`. It has no MQTT application API according to that profile. Native design/route evidence: `Eclipse-Arrowhead-go/ah5-core-go/serviceregistry/documentation`.

For this Java interface, select only a native transport and operation explicitly implemented and advertised by that port. A matching method/path does not establish DTO, error, authorization, retry or persistence equivalence. Java MQTT/AMQP bindings are not automatically provided by a Go or Rust role. Differential contract and security checks remain necessary before substitution.

5.2 Rust implementation

Primary system folder: `Eclipse-Arrowhead-rust/ah5-core-rust/serviceregistry`.

The per-system entry point is built by the `arrowhead-core-rust` crate. Build from the primary Rust workspace with the applicable package/binary selection. Documentation links can resolve into the shared core/support catalogue. Independent process identity and state remain required.

The ordinary native profile uses Tokio/Axum and rustls with peer-identity checks and independent locked JSON state. Network effects are not distributed database transactions. The precise trust, CRL reload and persistence guarantees are those of the native profile, not the Java implementation.

Native design/route evidence: `Eclipse-Arrowhead-rust/ah5-core-rust/documentation/ServiceRegistry`.

For this Java interface, select only a native transport and operation explicitly implemented and advertised by that port. A matching method/path does not establish DTO, error, authorization, retry or persistence equivalence. Java MQTT/AMQP bindings are not automatically provided by a Go or Rust role. Differential contract and security checks remain necessary before substitution.

6 References

[1] A. Banks and R. Gupta, “Mqtt version 3.1.1,” OASIS, Standard, 2014.

[2] E. Rescorla, “The transport layer security (tls) protocol version 1.3,” IETF, RFC 8446, 2018.

[3] T. Dierks and E. Rescorla, “The transport layer security (tls) protocol version 1.2,” IETF, RFC 5246, 2008.

[4] T. Bray, “The javascript object notation (json) data interchange format,” IETF, RFC 8259, 2017.

[5] J. Delsing, “deviceDiscovery – Service Description (ServiceRegistry),” Eclipse Arrowhead, SD 5.3.0, 2026, serviceRegistry-v5.3.0-deviceDiscovery-SD.pdf.

[6] ——, “deviceDiscovery HTTP/TLS/JSON – Interface Design Description (ServiceRegistry),” Eclipse Arrowhead, IDD 5.3.0, 2026, serviceRegistry-v5.3.0-deviceDiscovery-HTTP-IDD.pdf.

7 Revision History

7.1 Amendments

No.	Date	Version	Subject of Amendments	Author
1	2026-10-22	5.2.2	Document created from the Arrowhead LaTeX templates for the 5.2.2 Java Spring implementation, including the local extensions of this workspace.	Jerker Delsing
2	2026-11-09	5.3.0	New documentation edition; primary-source references and Go/Rust implementation scope; 5.2.2 retained.	Documentation maintenance

7.2 Quality Assurance

No.	Date	Version	Approved by
1		5.3.0	